Johns Hopkins Engineering

Introduction to GPU Programming

Module 8 Lecture 8A NVidia GPU-Accelerated Libraries – Solve FFTs using cuFFT

Material derived from https://developer.nvidia.com/cufft

JOHNS HOPKINS
WHITING SCHOOL
of ENGINEERING

JOHNS HOPKINS
WHITING SCHOOL
of ENGINEERING

Chancellor T. Pascale
Computer Science Department

Overview

1. Intro
2. Performance
3. Sample Code
4. Types
5. Data Layout

JOHNS HOPKINS
WHITING SCHOOL
of ENGINEERING

8.1 Intro

The material in this video is subject to the copyright of the owners of the material and is being provided for educational purposes under rules of fair use for registered students in this course only. No additional copies of the copyrighted work may be made or distributed.

8.1 Intro
8.2 Performance
8.3 Sample Code
8.4 Types
8.5 Data Layout

Intro

- “The NVIDIA CUDA Fast Fourier Transform library (cuFFT) provides a simple interface for computing FFTs up to 10x faster. By using hundreds of processor cores inside NVIDIA GPUs, cuFFT delivers the floating-point performance of a GPU without having to develop your own custom GPU FFT implementation.”

- “Widely used in applications ranging from computational physics to image processing and general signal processing, the Fast Fourier Transform is an efficient algorithm for computing discrete Fourier transforms of complex or real-valued data sets. cuFFT uses algorithms based on the well-known Cooley-Tukey and Bluestein algorithms, so you can be confident that you’re getting accurate results faster than ever.”

8.1 Intro
8.2 Performance
8.3 Sample Code
8.4 Types
8.5 Data Layout

Features

- 1D, 2D, 3D transforms of complex and real data types
- 1D transform sizes up to 128 million elements
- Flexible data layouts by allowing arbitrary strides between individual elements and array dimensions
- FFT algorithms based on Cooley-Tukey and Bluestein
- Familiar API similar to FFTW Advanced Interface
- Streamed asynchronous execution
- Single and double precision transforms
- Batch execution for doing multiple transforms
- In-place and out-of-place transforms
- Flexible input & output data layouts, similar to FFTW "Advanced Interface"
- Thread-safe & callable from multiple host threads”

JOHNS HOPKINS
WHITING SCHOOL
of ENGINEERING

8.2 Performance

The material in this video is subject to the copyright of the owners of the material and is being provided for educational purposes under rules of fair use for registered students in this course only. No additional copies of the copyrighted work may be made or distributed.

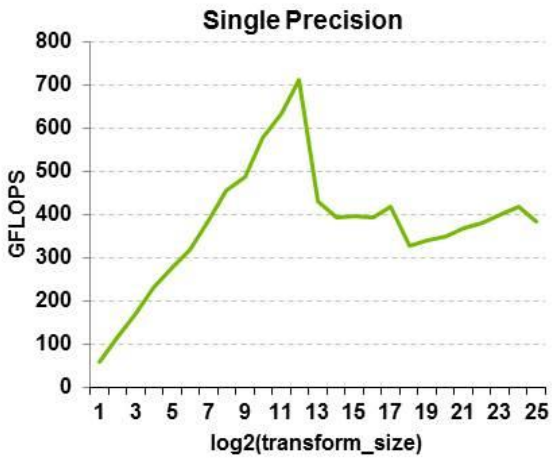
8.1 Intro
8.2 Performance
8.3 Sample Code
8.4 Types
8.5 Data Layout

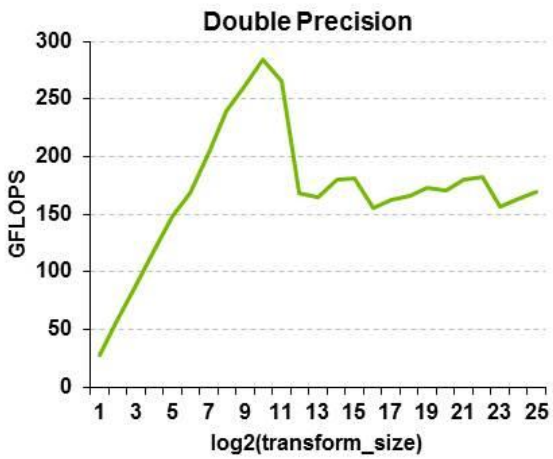
Performance

cuFFT: up to 700 GFLOPS

1D Complex, Batched FFTs

Used in Audio Processing and as a Foundation for 2D and 3D FFTs





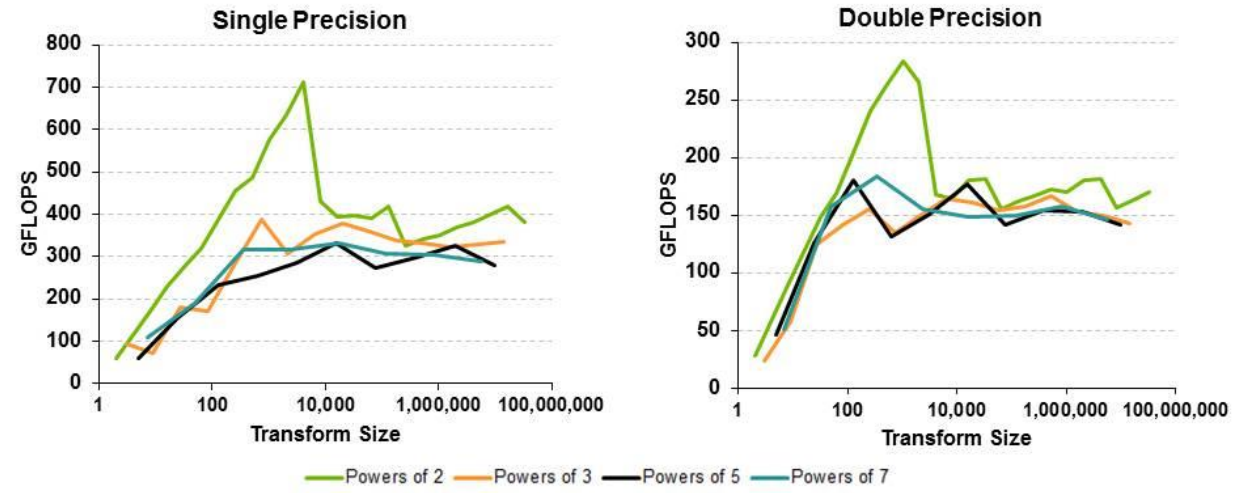
Performance may vary based on OS version and motherboard configuration

• cuFFT 6.0 on K40c, ECC ON, 32M elements, input and output data on device

8.1 Intro
8.2 Performance
8.3 Sample Code
8.4 Types
8.5 Data Layout

Performance

cuFFT: Consistently High Performance



Performance may vary based on OS version and motherboard configuration • cuFFT 6.0 on K40c, ECC ON, 28M-33M elements, input and output data on device

JOHNS HOPKINS
WHITING SCHOOL
of ENGINEERING

8.3 Sample Code

The material in this video is subject to the copyright of the owners of the material and is being provided for educational purposes under rules of fair use for registered students in this course only. No additional copies of the copyrighted work may be made or distributed.

8.1 Intro
8.2 Performance
8.3 Sample Code
8.4 Types
8.5 Data Layout

Sample Code

```
#define NX 64
#define NY 64
#define NZ 128
cufftHandle plan;
cufftComplex *data1, *data2;
cudaMalloc((void**)&data1, sizeof(cufftComplex)*NX*NY*NZ);
cudaMalloc((void**)&data2, sizeof(cufftComplex)*NX*NY*NZ);

/* Create a 3D FFT plan. */
cufftPlan3d(&plan, NX, NY, NZ, CUFFT_C2C);
/* Transform the first signal in place. */
cufftExecC2C(plan, data1, data1, CUFFT_FORWARD);
/* Transform the second signal using the same plan. */
cufftExecC2C(plan, data2, data2, CUFFT_FORWARD);

/* Destroy the cuFFT plan. */
cufftDestroy(plan);
cudaFree(data1); cudaFree(data2);
```

JOHNS HOPKINS
WHITING SCHOOL
of ENGINEERING

8.4 Types

The material in this video is subject to the copyright of the owners of the material and is being provided for educational purposes under rules of fair use for registered students in this course only. No additional copies of the copyrighted work may be made or distributed.

8.1 Intro
8.2 Performance
8.3 Sample Code
8.4 Types
8.5 Data Layout

Types

- cuFFT Plan Creation
  - Simple 1, 2, and 3-dimensional FF
    - cufftPlan1D() / cufftPlan2D() / cufftPlan3D()
  - Batched Input and complex data layouts
    - cufftPlanMany()
  - Like above supporting any type of precision
    - cufftXtMakePlanMany()

8.1 Intro
8.2 Performance
8.3 Sample Code
8.4 Types
8.5 Data Layout

Types

- cuFFT Plan Execution
  - Complex-to-Complex Single/Double Precision
    - cufftExecC2C() / cufftExecZ2Z()
  - Real-to-Complex Forward Transformation Single/Double Precision
    - cufftExecR2C() / cufftExecD2Z()
  - Real-to-Complex Inverse Transformation Single/Double Precision
    - cufftExecC2R() / cufftExecZ2D()

JOHNS HOPKINS
WHITING SCHOOL
of ENGINEERING

8.5 Data Layout

The material in this video is subject to the copyright of the owners of the material and is being provided for educational purposes under rules of fair use for registered students in this course only. No additional copies of the copyrighted work may be made or distributed.

8.1 Intro
8.2 Performance
8.3 Sample Code
8.4 Types
8.5 Data Layout

Data Layout

FFT type	input data size	output data size
C2C	x cufftComplex	x cufftComplex
C2R	x 2 + 1 cufftComplex	x cufftReal
R2C*	x cufftReal	x 2 + 1 cufftComplex

8.1 Intro
8.2 Performance
8.3 Sample Code
8.4 Types
8.5 Data Layout

Multidimensional Data Layout

Dims	FFT type	Input data size	Output data size
1D	C2C	N 1 cufftComplex	N 1 cufftComplex
1D	C2R	⌊ N 1 2 ⌋ + 1 cufftComplex	N 1 cufftReal
1D	R2C	N 1 cufftReal	⌊ N 1 2 ⌋ + 1 cufftComplex
2D	C2C	N 1 N 2 cufftComplex	N 1 N 2 cufftComplex
2D	C2R	N 1 (⌊ N 2 2 ⌋ + 1) cufftComplex	N 1 cufftReal
2D]	R2C	N 1 N 2 cufftReal	N 1 (⌊ N 2 2 ⌋ + 1) cufftComplex
3D	C2C	N 1 N 2 N 3 cufftComplex	N 1 N 2 N 3 cufftComplex
3D	C2R	N 1 N 2 (⌊ N 3 2 ⌋ + 1) cufftComplex	N 1 N 2 N 3 cufftReal
3D	R2C	N 1 N 2 N 3 cufftReal	N 1 N 2 (⌊ N 3 2 ⌋ + 1) cufftComplex